Fullstack Take Home Assignment

High-Throughput Inventory Reservation System & Mini Dashboard

TARGET LEVEL	STRICT TECH STACK	TIME COMMITMENT
Mid–Senior Fullstack	**Go + Gin + PostgreSQL | React + Vite**	**4–8 Hours Max**

🎯 1. Assignment Objective

During flash-sale events, thousands of concurrent users attempt to purchase high-demand products simultaneously. The objective of this assessment is to evaluate your ability to design a resilient, high-concurrency backend service using **Go, Gin Framework, and a real PostgreSQL database** (80% weighting) and connect it to a clean, functional frontend application built strictly using **React and Vite** (20% weighting).

📡 2. Backend Specifications & Mandatory Stack (80% Weighting)

Your backend microservice must be built using **Go (Golang)** with the **Gin HTTP Web Framework** (`github.com/gin-gonic/gin`). You must use a **real PostgreSQL database instance running via Docker Compose** (in-memory databases, SQLite, or mock data stores are strictly disallowed).

Your service must expose the following REST HTTP endpoints. For each endpoint, implement both successful and negative error scenarios, designing the JSON response structure that best fits API best practices:

1. Reserve Stock

- **Endpoint:** `POST /api/v1/inventory/reserve`
- **Objective:** Atomically reserve inventory in PostgreSQL for a user if sufficient available stock exists. Successfully reserved stock must automatically expire after **5 minutes** unless confirmed.
- **Request Payload:**

```
{
  "user_id": "usr_9981",
  "item_id": "item_4021",
  "quantity": 2
}
```

- **Response Payload (Success):**

```
{
  "status": "success",
  "reservation_id": "res_883291",
  "item_id": "item_4021",
  "quantity": 2,
  "expires_at": "2026-07-20T16:35:00Z"
}
```

- **Negative Cases:** Handle and implement negative error scenarios (e.g., insufficient inventory, invalid input parameters) and design the JSON response structure you deem best for client integration.

2. Confirm Reservation

- **Endpoint:** `POST /api/v1/inventory/confirm`

- **Objective:** Permanently commit an active reservation in PostgreSQL, completing the order process and decrementing physical inventory.
- **Request Payload:**

```
{
  "reservation_id": "res_883291"
}
```

- **Response Payload (Success):**

```
{
  "status": "success",
  "reservation_id": "res_883291",
  "confirmed_at": "2026-07-20T16:32:00Z"
}
```

- **Negative Cases:** Handle and implement negative error scenarios (e.g., expired reservations, non-existent reservation IDs) and design the JSON response structure you deem best for client integration.

3. Check Inventory Status

- **Endpoint:** `GET /api/v1/inventory/stock?item_id=item_4021`
- **Objective:** Query PostgreSQL for real-time stock breakdowns for an item. Handle negative/edge cases where the item is not found or parameters are invalid, designing appropriate error JSON structures.
- **Response Payload (Success):**

```
{
  "item_id": "item_4021",
  "total_stock": 100,
  "reserved_stock": 15,
  "available_stock": 85
}
```

🖥️ 3. Frontend Specifications — Strict React + Vite Requirement (20% Weighting)

The frontend mini-dashboard **must be built strictly using React initialized with Vite** (JavaScript or TypeScript). Other frameworks (Vue, Svelte, Angular, Next.js, Nuxt) or plain HTML/JS solutions are **strictly unacceptable** and will result in submission rejection.

Implement the following core user interface workflows:

1. **Live Inventory Tracker:** Display current total, reserved, and available stock for an item with a manual refresh button or automated polling.
2. **Reservation Form:** A clean form allowing a user to input a User ID and Quantity to request a reservation from the Gin backend.
3. **Expiration & Confirmation Workflow:**
   - Show active reservation details alongside a live 5-minute countdown timer.
   - Provide a **"Confirm Purchase"** button to execute order confirmation.
   - Gracefully handle and render negative backend response states (e.g., failed reservations, expired sessions) directly on the React UI.

⚙️ 4. Mandatory Technical Requirements & Quality Expectations

1. **Strict Stack Enforcement:**
   - **Backend Framework:** Go with Gin Framework.

- **Database Store:** Real PostgreSQL instance. You must include SQL migration files or schema initialization scripts.
- **Frontend Framework:** React + Vite strictly.
- **Mandatory Docker Setup:** You must provide a working `docker-compose.yaml` file that spins up the PostgreSQL container alongside the backend (and optionally the frontend) so the entire system can be run with a single command (e.g., `docker compose up`).

2. **Data Integrity & Concurrency:** Strict PostgreSQL transaction handling and concurrency control. Over-selling or negative stock allocations under heavy concurrent load are strictly unacceptable.
3. **Automatic Expiry Cleanup:** Expired reservations must be automatically returned to available stock upon expiration.
4. **Graceful Shutdown:** The Gin server must cleanly intercept system signals and shut down gracefully without corrupting state or dropping active database connections.
5. **Backend Testing:** Include unit and stress tests. The Go codebase will be validated using standard race detection:

```
go test -race -v ./...
```

📝 5. Required Written Deliverable: `ARCHITECTURE.md`

Address the following three prompts in an `ARCHITECTURE.md` file (1–2 pages max):

1. **Architectural Design & Database Locking Strategy:** Defend your PostgreSQL schema design, Gin routing structure, and database synchronization/transaction management choices for high concurrency.
2. **Distributed Scaling & Failure Modes:** What breaks if scaled horizontally across 10 service instances pointing to PostgreSQL? How would you redesign it for a stateless, distributed deployment?
3. **Engineering Trade-offs & AI Transparency:** Describe trade-offs made within the 4–8 hour window across both React and Go stacks. If you used AI tools (ChatGPT, Copilot, Claude), describe one scenario where an AI suggestion was flawed and how you fixed it.

📦 6. Submission Guidelines & Git Requirements

Proof of Work (Incremental Git Commit History):

- Submit a link to a private/public Git repository (GitHub/GitLab). **No zip files or single-commit code dumps.**
- Commit incrementally across both React frontend and Go backend development phases. Submissions with a single massive commit will be rejected.

1. **Setup Instructions:** Include a `README.md` detailing how to run PostgreSQL via `docker compose up`, start the Gin backend, and run the React + Vite frontend.

Next Stage Notice: Passing submissions will advance to a **30-minute live code walkthrough & defense call** where you will present your React & Go implementation and discuss your architectural choices.